Sustentación Tecnológica: Lenguaje y Base de Datos

Este documento detalla los criterios técnicos y arquitectónicos que fundamentan la elección del stack tecnológico (React, TypeScript y Supabase) empleado para la construcción del ecosistema digital móvil de HumedApp.

---

1. Lenguaje de Programación y Tecnologías Frontend (React + TypeScript)

1.1 ¿Qué criterios usó para seleccionar el lenguaje de programación?

El criterio principal fue la necesidad de construir una aplicación multiplataforma (accesible desde móviles y computadoras) de forma ágil, sin requerir el mantenimiento de múltiples bases de código. Se priorizó un lenguaje y un framework que permitiera crear un cliente dinámico (*Single Page Application* - SPA) capaz de conectarse sin problemas a APIs asíncronas para el uso del GPS nativo (Leaflet) y la inteligencia artificial (Google Gemini). Adicionalmente, se exigió un lenguaje que proveyera seguridad estructural, para lo cual se adoptó TypeScript, que dota a JavaScript de tipado estático, previniendo errores en tiempo de ejecución.

1.2 ¿Qué factores influyeron más en su decisión?

1. **Facilidad de uso:** La arquitectura basada en Componentes de React permite dividir la interfaz en piezas modulares y manejables (como el `Dashboard`, `Chatbot` o `Mapa`), facilitando el desarrollo y la reutilización de código.
2. **Experiencia previa:** La curva de aprendizaje es más corta cuando se cuenta con conocimientos previos en ecosistemas web y JavaScript, agilizando el ciclo de desarrollo del proyecto grado.
3. **Documentación:** React y TypeScript poseen la comunidad open-source más grande del mundo. Existen bibliotecas con soporte y documentación profunda, tales como `Tailwind CSS` y `@supabase/supabase-js`.
4. **Demanda laboral:** El stack React/TypeScript es actualmente el estándar de la industria web moderna, aportando peso y profesionalismo al perfil del graduando.
5. **Otros (Herramientas):** El uso del empaquetador moderno **Vite**, que mejora radicalmente los tiempos de carga en desarrollo (*Hot Module Replacement*) en comparación a tecnologías nativas más lentas.

1.3 ¿Qué otras alternativas de lenguaje consideró y por qué las descartó?

- **Java/Kotlin (Android Nativo) o Swift (iOS):** Se descartaron rápidamente debido al costo temporal. Requerían escribir y mantener lógicas completamente separadas para cada ecosistema móvil, cuando los requerimientos de la aplicación podían resolverse elegantemente a través de un ecosistema progresivo multiplataforma (PWA).
- **Python (a través de frameworks como Django):** Aunque es excelente para backends complejos, su desarrollo para renderizado de interfaces enriquecidas en cliente carece de la fluidez y dinamismo que ofrece el ecosistema JavaScript sin recargar las pantallas constantemente.
- **Dart (Flutter):** Se consideró por ser multiplataforma, pero se descartó para aprovechar la sólida base de recursos JavaScript ya existente para mapas web (`Leaflet`) y por la inmediatez en su aprendizaje e integración con la IA de Google.

1.4 ¿Qué ventajas encontró en el lenguaje seleccionado frente a las otras opciones?

La ventaja diferencial más importante fue la **velocidad de iteración e integración**. Con React y TypeScript los módulos se comunicaron perfectamente con la cámara del dispositivo y las API REST (Gemini y Supabase) unificando la lógica de negocio en un solo repositorio. Además, el tipado estricto de TypeScript

(ej., la interfaz `EnvironmentalReport`) aseguró que el formato de los datos que se guardaban en la base de datos coincidiera perfectamente en todo el flujo, evitando errores impredecibles durante el uso ciudadano.

---

2. Base de Datos (PostgreSQL mediante Supabase)

2.1 ¿Qué criterios tuvo en cuenta para su base de datos?

La base de datos requerida debía integrar capacidad relacional estricta (ya que los reportes de flora/fauna están vinculados a usuarios específicos), soporte para registrar coordenadas geográficas, y crucialmente, la capacidad de evitar la codificación de un backend robusto desde cero. Se buscó una solución del tipo *Backend-as-a-Service* (BaaS) con sistema de autenticación "out-of-the-box".

2.2 ¿Qué tipo de base de datos seleccionó y por qué es la más adecuada?

Se seleccionó la base de datos relacional y de código abierto **PostgreSQL**, servida a través de **Supabase**. Es la más adecuada porque cumple los principios ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), limitando la duplicación de datos (integridad referencial). Al mismo tiempo, el servicio expone SDKs que permiten que la app en React consulte o altere datos en tiempo real sin requerir construir o alojar un servidor intermedio pesado.

2.3 ¿Qué otras bases de datos evaluó antes de tomar la decisión final?

- **Firebase (Firestore / NoSQL):** Su modelo no-relacional obliga a construir estructuras redundantes complejas cuando se intentan asociar perfiles de usuario comunitarios con denuncias que requieren validaciones relacionales cruzadas (`JOINs`). También genera *Vendor Lock-In* (dependencia cerrada del ecosistema de Google).
- **MySQL / MongoDB (Local o Self-Hosted):** Se descartaron porque implicaban gastar semanas montando una API intermediaria completa (Node.js/Express) sólo para conectar la app React con la base de datos, lidiar con la encriptación manual y gestionar tokens de seguridad, desviando el objetivo principal del proyecto de grado ambiental.

2.4 Características técnicas de la base de datos

- **Tipo:** Base de Datos Relacional / Objeto-Relacional (PostgreSQL V15+).
- **Escalabilidad y Límites Operativos:** Soporta conexiones altamente concurrentes (connection pooler), procesando sin fricción cientos de miles de registros en su umbral introductorio gratuito. Brinda 500 MB exclusivos para el modelo relacional puro, e independientes gigabytes para medios masivos como imágenes.
- **Migración: SÍ permite migración.** Supabase es una extensión directa sobre PostgreSQL de código abierto. Se puede ejecutar un `pg_dump` tradicional para resguardar toda la estructura y datos y moverlos a cualquier servidor físico propio, Amazon RDS o Cloud SQL en cuestión de minutos.

2.5 ¿Qué ventajas ofrece frente a las otras bases de datos?

La fusión del motor PostgreSQL robusto con la interfaz Supabase provee en su capa superior WebSockets por defecto; así, el Front-End (React) se suscribe y escucha la inserción de nuevas denuncias en tiempo real, vital para mostrar pines inmediatamente en el mapa. Proveen la autogeneración de la API de datos y la administración de la Autenticación de forma gratuita e integral sin configuraciones engorrosas de servidor apache/nginx.

2.6 ¿Cómo va a manejar la seguridad de la base de datos? (Protección y enmascaramiento)

1. **Row Level Security (RLS) Nativo:** Las políticas RLS a nivel del motor impiden, incluso si un atacante conoce el endpoint, acceder, borrar o editar un reporte a menos que el validador estricto lo reconozca como el usuario autenticado (dueño del reporte) bajo el token JWT en el lado del servidor.
2. **Encriptación Integral:** Toda petición viaja bajo estándares seguros SSL/TLS, protegiendo las identidades mediante el hash encriptado `bcrypt/argon2`.
3. **Manejo de Llaves Locales:** Todas las URLs públicas de conectividad al servidor SQL han sido enmascaradas en un archivo de entorno protegido (`.env.local`) de Vite para prevenir inyecciones de código.

---

3. Justificación de Supabase Storage vs MySQL para Almacenamiento de Imágenes

¿Por qué es superior Supabase Storage frente a usar un BLOB directamente sobre un motor tradicional como MySQL para guardar las fotos de los humedales?

3.1. Separación de Responsabilidades (Object Storage vs. BLOBs)

- **El problema con MySQL:** Obliga a convertir la imagen a código binario grueso y masivo (`BLOB`). Esto contamina la base de datos relacional provocando consultas extremadamente lentas y copias de seguridad de terabytes.
- **La solución (Supabase):** Emplea `Supabase Storage` diseñado específicamente para medios masivos. Las fotos se suben allí, y en la tabla de base de datos *solo se ingresa una cadena de texto ultraligera de la URL generada*.

3.2. Eliminación de Backend intermedio

- **El problema con MySQL:** Demandaría programar, por meses, un API en Node o Python exclusiva para tomar el formulario *multipart-formData* cargado desde el móvil, desempaquetarlo, analizarlo por seguridad e insertarlo manualmente en MySQL.
- **La solución (Supabase):** React, usando la librería cliente ya proporcionada, sube el archivo directo desde el celular a los contenedores de AWS/S3 internos mediante la instrucción mágica: `supabase.storage.from('reportes').upload()`.

3.3 CDN e Infraestructura Global

- **El problema con MySQL:** Si 50 residentes acceden al Dashboard intentando descargar las fotografías directamente desde registros BLOB de MySQL locales al mismo tiempo, la red saturada ahogaría cualquier servidor estudiantil local.
- **La solución (Supabase):** Las imágenes subidas transitan a través de una red CDN optimizada globalmente, la cual entrega los archivos milisegundos más rápido operando independientemente de la carga de la base de datos para asegurar rendimiento móvil.

3.4 Optimización automática Inteligente

- Si una imagen cargada pesa originalmente 10 MB, extraer ese pesado BLOB desde MySQL destruiría el plan de datos y la paciencia del ciudadano promedio. Supabase provee la capacidad de "*Transformación en vuelo*"; podemos dictar parámetros en la URL ordenándole que devuelva versiones comprimidas en un *formato veloz como WebP de 100x100 píxeles de anchura*, ideal para los iconos miniatura flotantes en el mapa sobre las denuncias.